

CSE332: Computer Architecture and Organization

Assignment#4

Section - 5

Submitted by,

Name: Muhtadina Serniabat Tasin

Student ID: 2232160642

Submitted to,

Faculty: MAQm

Submission Date: 11 April 2025

Spring Semester 2025
North South University

1. A 32 bit ALU in Verilog with OR, AND, NOR, ADD, SUB, SLT logic

a. Verilog program for the 32 bit ALU:

```
full_adder_en.v
// === mux2x1.v ===
module mux2x1 (input A, B, Sel, output Q);
    assign Q = Sel ? B : A;
endmodule

// === mux4x1.v ===
module mux4x1 (
    input A, B, C, D,
    input [1:0] Sel,
    output Q
);
    assign Q = (Sel == 2'b00) ? A :
               (Sel == 2'b01) ? B :
               (Sel == 2'b10) ? C : D;
endmodule

// === full_adder_en.v ===
module full_adder_en (
    input A, B, Cin,
    input Ainvert, Binvert,
    input [1:0] Op,
    input Less,
    output Result, Cout
);
    wire A_in, B_in;
    wire sum, and_out, or_out;

    assign A_in = Ainvert ? ~A : A;
    assign B_in = Binvert ? ~B : B;

    assign and_out = A_in & B_in;
    assign or_out = A_in | B_in;
    assign sum = A_in ^ B_in ^ Cin;
    assign Cout = (A_in & B_in) | (A_in & Cin) | (B_in & Cin);

    mux4x1 mux_op (
        .A(and_out),
        .B(or_out),
        .C(sum),
        .D(Less),
        .Sel(Op),
    );
endmodule
```

```

        .Q(Result)
    );
endmodule

```

alu32.v

```

module alu32 (
    input [31:0] A,
    input [31:0] B,
    input [2:0] ALUControl,
    output [31:0] Result,
    output Cout, Zero
);
    wire [31:0] Carry;
    wire [31:0] R;
    wire Binvert = (ALUControl == 3'b110 || ALUControl == 3'b111);
    wire Ainvert = 0;
    wire set;

    full_adder_en FA0 (
        .A(A[0]), .B(B[0]), .Cin(Binvert),
        .Ainvert(Ainvert), .Binvert(Binvert),
        .Op(ALUControl[1:0]),
        .Less(1'b0), .Result(R[0]), .Cout(Carry[0])
    );

    genvar i;
    generate
        for (i = 1; i < 31; i = i + 1) begin : full_adders
            full_adder_en FA (
                .A(A[i]), .B(B[i]), .Cin(Carry[i-1]),
                .Ainvert(Ainvert), .Binvert(Binvert),
                .Op(ALUControl[1:0]),
                .Less(1'b0), .Result(R[i]), .Cout(Carry[i])
            );
        end
    endgenerate

    wire Cout_MSB;
    wire SetFlag = (A[31] ^ B[31]) ? A[31] : Carry[30];
    full_adder_en FA_MSB (
        .A(A[31]), .B(B[31]), .Cin(Carry[30]),
        .Ainvert(Ainvert), .Binvert(Binvert),
        .Op(ALUControl[1:0]),
        .Less(SetFlag), .Result(R[31]), .Cout(Cout_MSB)
    );

    assign Result = (ALUControl == 3'b100) ? ~(A | B) : R;
    assign Cout = Cout_MSB;

```

```
assign Zero = (Result == 32'b0);  
endmodule
```

b. Types of Verilog Modeling Used:

1. Structural Modeling

Used **structural modeling** to **build the 32-bit ALU from smaller modules** (like 1-bit ALUs, multiplexers). Structural Verilog is essentially like drawing a circuit diagram using code — it's all about **connecting instances of other modules together**.

Examples in your design:

- Instantiating `full_adder_en` 32 times to build the 32-bit ALU.
- Instantiating `mux2x1` and `mux4x1` inside `full_adder_en`.

It provides a clear, hierarchical way to build complex systems from simple blocks, which is great for understanding and debugging.

2. Behavioral Modeling

Used inside the `mux4x1` and `mux2x1` modules — these use `assign` statements and `always @(*)` to describe what the circuit should do.

Examples:

- `assign Q = (Sel == 2'b00) ? A :`
- `(Sel == 2'b01) ? B :`
- `(Sel == 2'b10) ? C : D;`

Multiplexer behavior is simple and easily described in one line; no need to break them down into gates.

3. Hierarchical Design

The entire ALU design is **hierarchical**, meaning that:

- `alu32` is the **top-level** module,
- It instantiates 32 copies of `full_adder_en`,
- Each `full_adder_en` uses `mux4x1`, `mux2x1`.

Improves modularity, readability, testing, and reusability. You can debug/test each module independently.

full_adder_en.v

```
module full_adder_en (
    input A, B, Cin,          // Single-bit inputs and carry-in
    input Ainvert, Binvert,    // Control signals to invert A and/or B (used for SUB)
    input [1:0] Op,           // Operation select: 00=AND, 01=OR, 10=ADD, 11=SLT
    input Less,               // For SLT operation, used only in LSB
    output Result, Cout       // Single-bit result and carry out
);
    // Internal logic signals
    wire A_in, B_in;
    wire sum, and_out, or_out;

    // Invert inputs if control signals are set
    assign A_in = Ainvert ? ~A : A;
    assign B_in = Binvert ? ~B : B;

    // Logic operations
    assign and_out = A_in & B_in;
    assign or_out = A_in | B_in;
    assign sum = A_in ^ B_in ^ Cin;

    // Carry-out logic for full adder
    assign Cout = (A_in & B_in) | (A_in & Cin) | (B_in & Cin);

    // 4-to-1 multiplexer to choose final output
    mux4x1 mux_op (
        .A(and_out), .B(or_out), .C(sum), .D(Less),
        .Sel(Op),
        .Q(Result)
    );
endmodule
```

alu32.v

```
module alu32 (
    input [31:0] A, B,        // 32-bit input operands
    input [2:0] ALUControl,    // Operation selector (AND, OR, ADD, SUB, SLT, etc.)
    output [31:0] Result,      // 32-bit result of the ALU
);
```

```

output Cout,          // Final carry out (useful for overflow detection)
output Zero           // Flag to indicate if result is zero
);
wire [31:0] Carry;    // Carry wires between 1-bit ALUs
wire [31:0] R;        // Internal result before final output assignment

wire Binvert = (ALUControl == 3'b110 || ALUControl == 3'b111); // SUB or SLT
wire Ainvert = 0;

// LSB: connect carry-in and process Less (SLT flag)
full_adder_en FAo (
    .A(A[0]), .B(B[0]), .Cin(Binvert),
    .Ainvert(Ainvert), .Binvert(Binvert),
    .Op(ALUControl[1:0]),
    .Less(1'bo), .Result(R[0]), .Cout(Carry[0])
);

// Middle bits: repeated logic
genvar i;
generate
    for (i = 1; i < 31; i = i + 1) begin : full_adders
        full_adder_en FA (
            .A(A[i]), .B(B[i]), .Cin(Carry[i-1]),
            .Ainvert(Ainvert), .Binvert(Binvert),
            .Op(ALUControl[1:0]),
            .Less(1'bo), .Result(R[i]), .Cout(Carry[i])
        );
    end
endgenerate

// MSB: Set logic for SLT
wire Cout_MSB;
wire SetFlag = (A[31] ^ B[31]) ? A[31] : Carry[30];
full_adder_en FA_MSB (
    .A(A[31]), .B(B[31]), .Cin(Carry[30]),
    .Ainvert(Ainvert), .Binvert(Binvert),
    .Op(ALUControl[1:0]),
    .Less(SetFlag), .Result(R[31]), .Cout(Cout_MSB)
);

assign Result = (ALUControl == 3'b100) ? ~(A | B) : R; // Support NOR
assign Cout = Cout_MSB;
assign Zero = (Result == 32'bo); // Useful for branching
endmodule

```

c. Testbench for the 32 bit ALU - alu32_tb.v

alu32_tb.v

```
`timescale 1ns / 1ps

module alu32_tb;

    // Inputs
    reg [31:0] A;
    reg [31:0] B;
    reg [2:0] ALUControl;

    // Outputs
    wire [31:0] Result;
    wire Cout;
    wire Zero;

    // Instantiate the ALU
    alu32 uut (
        .A(A),
        .B(B),
        .ALUControl(ALUControl),
        .Result(Result),
        .Cout(Cout),
        .Zero(Zero)
    );

    // Task to display results
    task display_output;
    begin
        $display("ALUControl = %b | A = %h | B = %h | Result = %h | Cout = %b | Zero
= %b",
                ALUControl, A, B, Result, Cout, Zero);
    end
endtask

initial begin
    $display("---- ALU 32-bit Testbench ----");

    // Test AND
    A = 32'hFFFF0000;
    B = 32'hoFoFoFoF;
    ALUControl = 3'b000;
    #10 display_output;

    // Test OR
    A = 32'hFFFF0000;
    B = 32'hoFoFoFoF;
    ALUControl = 3'b001;
```

```

#10 display_output;

// Test ADD
A = 32'd100;
B = 32'd200;
ALUControl = 3'b010;
#10 display_output;

// Test SUB
A = 32'd500;
B = 32'd200;
ALUControl = 3'b110;
#10 display_output;

// Test SLT (A < B)
A = 32'd50;
B = 32'd100;
ALUControl = 3'b111;
#10 display_output;

// Test SLT (A > B)
A = 32'd200;
B = 32'd100;
ALUControl = 3'b111;
#10 display_output;

// Test ADD with overflow
A = 32'h7FFFFFFF;
B = 32'd1;
ALUControl = 3'b010;
#10 display_output;

// Test SUB to get negative result
A = 32'd100;
B = 32'd200;
ALUControl = 3'b110;
#10 display_output;

// Randomized ALU ops (5 types only)
repeat(5) begin
    A = $random;
    B = $random;
    ALUControl = $urandom_range(0, 4) == 4 ? 3'b111 : 3'b000 +
$urandom_range(0, 3); // restrict to 000, 001, 010, 110, 111
    #10 display_output;
end

$display("---- Testbench Complete ----");
$finish;
end

```

endmodule

d. Outputs:

